



TRABAJO DE FINAL DE GRADO

GRADO EN INTELIGENCIA ROBÓTICA

Estudio e implementación del control de péndulo invertido sobre carro en un sistema de tiempo real

Estudiante:
Miguel Porcar Vicent

Tutor:
Ignacio Peñarrocha Alós

Fecha de presentación: Junio de 2026
Curso académico: 2025/2026

Agradecimientos:

Palabras clave:

CASTELLÓN, a XX de XXXXXXX de 2025

Todos los derechos reservados

ÍNDICE GENERAL

1	Introducción	5
1.1	Motivaciones	5
1.2	Objetivos	5
1.3	Planificación	5
1.4	Estimación de costes	5
2	Análisis	7
2.1	Requisitos funcionales	8
2.2	Requisitos no funcionales	9
2.3	Diseño del sistema	10
2.3.1	Introducción al modelado dinámico	10
2.3.2	Modelo dinámico del sistema	11
2.3.3	Estrategia de control Swing-up	13
2.3.4	Modelado en el espacio de estados	16
2.3.5	Control por realimentación del estado	18
2.3.6	Lógica de conmutación	19
2.4	Arquitectura del sistema	20
2.4.1	Arquitectura de la simulación	20
2.4.2	Arquitectura del sistema real	21
2.4.3	Especificación de componentes de hardware	22
3	Desarrollo	25
3.1	Simulación	25
3.2	Sistema físico	25

4	Resultados	27
4.1	Resultados en simulación	27
4.2	Resultados en sistema físico	27
5	Conclusiones y posible trabajo futuro	29
	Bibliografía	31
6	Apéndices	33
A	Desarrollo del Lagrangiano	35

1. Introducción

1.1 Motivaciones

En caso de que haya motivo(s) concreto(s) para la realización del proyecto.

1.2 Objetivos

Objetivos que se espera alcanzar al inicio del proyecto.

1.3 Planificación

Lista de tareas, relación entre ellas y duración estimada Posibles desviaciones de la planificación Diagrama de Gantt

1.4 Estimación de costes

Estimación del coste económico que supondría la adquisición del material necesario (hardware y software) para la realización del proyecto, los salarios involucrados y otros aspectos como alquileres de espacios y energía.

2. Análisis

En este capítulo se realiza un estudio detallado de los pilares que sustentan el desarrollo del sistema de control para el péndulo invertido. El análisis se articula en tres ejes fundamentales:

- **Análisis de requisitos:** Donde se formalizan las capacidades operativas (funcionales) y las restricciones técnicas, de seguridad y rendimiento (no funcionales) que debe satisfacer el sistema.
- **Diseño del sistema:** Se describe la estrategia lógica y matemática adoptada, detallando el modelo dinámico, las leyes de control híbridas y la lógica de conmutación necesaria para el *swing-up* y la estabilización.
- **Arquitectura del sistema:** Se define la infraestructura tanto de simulación como de implementación real, especificando el flujo de datos, la integración con ROS y la selección de los componentes de hardware que garantizan la viabilidad del proyecto.

2.1 Requisitos funcionales

Los requisitos funcionales describen las acciones, cálculos y manipulaciones de datos específicos que el sistema debe realizar para cumplir su propósito. Para el control del péndulo invertido sobre carro se definen mediante sus entradas, comportamiento y salidas:

■ RF1. Modelado y simulación de la dinámica:

- *Entradas:* Parámetros físicos del sistema (masas, inercias, coeficientes de fricción) y señal de control simulada.
- *Comportamiento:* Resolución matemática de las Ecuaciones Diferenciales Ordinarias (EDO) que rigen la dinámica no lineal del mecanismo en el entorno MATLAB/Simulink.
- *Salidas:* Estado cinemático simulado del sistema (posiciones y velocidades articulares).

■ RF2. Visualización y telemetría en ROS:

- *Entradas:* Estado cinemático del sistema (proveniente de la simulación o del hardware real) y archivo de descripción geométrica URDF.
- *Comportamiento:* Traducción y publicación de los datos mediante el puente de comunicación de ROS Toolbox para actualizar el árbol de transformadas (TF) del robot.
- *Salidas:* Visualización 3D actualizada en tiempo real mediante la interfaz RViz (tópico `/joint_states`).

■ RF3. Control de inyección de energía (Swing-up):

- *Entradas:* Posición y velocidad angular actual del péndulo ($q, \dot{q}$).
- *Comportamiento:* Cálculo y ejecución de una ley de control basada en energía para balancear el péndulo desde su posición estable ($q = \pi$ rad) hasta el punto de linealización del sistema ($q = 0$ rad).
- *Salidas:* Acción de control (u).

■ RF4. Control de estabilización (Realimentación del estado):

- *Entradas:* Vector de estado del sistema ($x = [p, \dot{p}, q, \dot{q}]^T$).
- *Comportamiento:* Estabilización del sistema sobre el punto de linealización utilizando control por realimentación del estado.
- *Salidas:* Acción de control (u).

■ RF5. Adquisición y decodificación de señales:

- *Entradas:* Señales eléctricas de pulsos en cuadratura (A y B) provenientes de los encoders del motor y del péndulo.
- *Comportamiento:* Procesamiento y conteo de los pulsos digitales mediante el hardware específico.
- *Salidas:* Valores digitales discretos de posición y velocidad interpretables por la ley de control.

2.2 Requisitos no funcionales

Los requisitos no funcionales imponen condiciones y restricciones a la implementación del diseño:

- **RNF1. Restricciones de rendimiento (Determinismo y tiempo real):** El bucle de control debe ejecutarse dentro de una rutina de interrupción periódica en un microcontrolador, garantizando un periodo de muestreo estable y un *jitter* despreciable para la viabilidad del control discreto.
- **RNF2. Restricciones de fiabilidad (Robustez):** El sistema debe rechazar perturbaciones externas moderadas y ser tolerante a las discrepancias producidas por dinámicas no modeladas (fricciones no lineales y holguras mecánicas).
- **RNF3. Restricciones de seguridad (Gestión de fallos):**
 - *Seguridad mecánica:* El software debe implementar una parada de emergencia si las lecturas de posición indican que el carro excede los límites físicos del riel. En caso de fallo en el software de parada se debe disponer de finales de carrera para cortar la alimentación del actuador.
 - *Seguridad eléctrica:* Se debe garantizar el aislamiento galvánico entre la etapa de control (3.3V) y la etapa de potencia (24V).
- **RNF4. Condiciones de implementación (Abstracción HAL):** El código desarrollado debe hacer uso de la capa de abstracción de hardware (HAL) ofrecida por el fabricante del microcontrolador, asegurando la portabilidad, estandarización y eficiencia del firmware.

2.3 Diseño del sistema

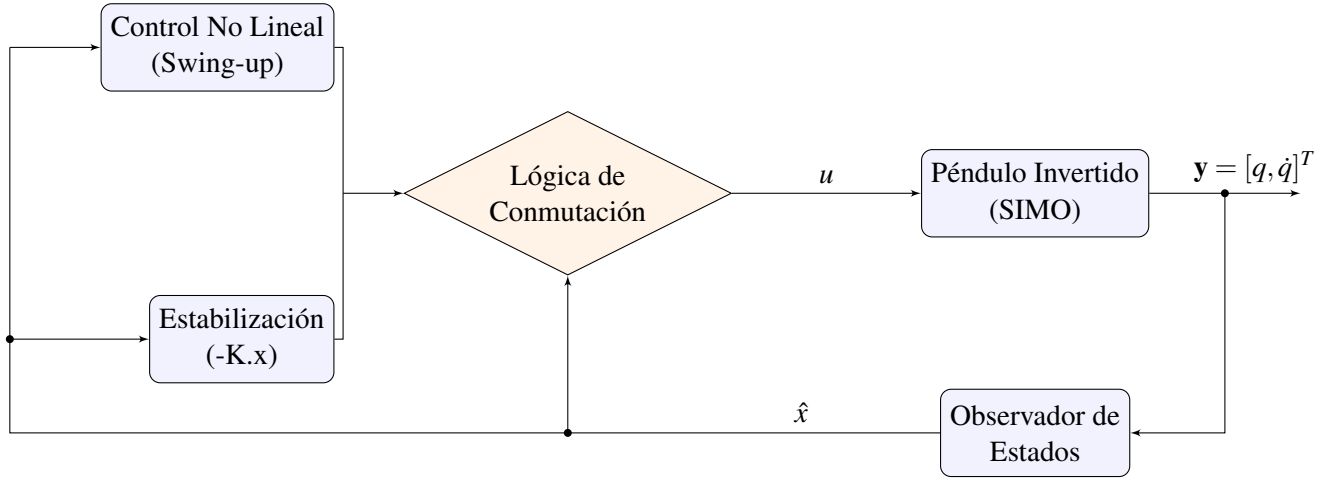


Figura 2.1: Arquitectura de control

La arquitectura de control propuesta en la Figura 2.1 se aplica sobre un sistema subactuado tipo SIMO (Single Input Multiple Output). Para la etapa de realimentación, se implementa un observador del estado por inversión. Mediante este enfoque, los estados del sistema se obtienen exclusivamente a partir de las mediciones de salida, dado que, en el caso concreto de sistemas robóticos, el estado coincide con las salidas cinemáticas $(q, \dot{q})$.

2.3.1 Introducción al modelado dinámico

A diferencia de la cinemática, que estudia el movimiento de un mecanismo sin considerar las fuerzas y pares que lo producen, el modelado dinámico describe explícitamente la relación entre la fuerza aplicada y el movimiento resultante [1]. Disponer de las ecuaciones de movimiento exactas es un requisito estricto y fundamental tanto para la simulación del sistema como para el diseño de controladores basados en modelo.

Para determinar la dinámica del péndulo invertido sobre carro, se recurre a las ecuaciones de Euler-Lagrange. Este enfoque energético se basa en la obtención del Lagrangiano del sistema (L), definido como la diferencia entre la energía cinética (E_c) y la energía potencial (E_p):

$$L = E_c - E_p \quad (2.1)$$

Las ecuaciones diferenciales no lineales resultantes pueden reescribirse y compactarse de forma elegante en la formulación matricial clásica para sistemas robóticos [2]:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = Bu \quad (2.2)$$

Para el sistema en concreto, las dimensiones de cada matriz son:

- $M(q) \in \mathbb{R}^{2 \times 2}$ es la matriz de inercia.
- $C(q, \dot{q}) \in \mathbb{R}^{2 \times 2}$ es la matriz de fuerzas centrípetas y de Coriolis.
- $G(q) \in \mathbb{R}^{2 \times 1}$ es el vector de fuerzas gravitacionales.
- $B \in \mathbb{R}^{2 \times 1}$ vector de entrada, mapea la acción de control u .

Este enfoque matricial no solo facilita el análisis teórico, sino que permite realizar correcciones en la simulación del sistema muy fácilmente, ya que los parámetros que lo caracterizan (masas, longitudes, inercias, etc.) se encapsulan dentro de las propias matrices.

2.3.2 Modelo dinámico del sistema

El primer paso para el modelado dinámico consiste en definir un sistema de referencia en el plano bidimensional (x, y) . Esto permite localizar la posición del centro de masas (CM) del péndulo en función de las coordenadas generalizadas del sistema: la posición lineal del carro (x) y la posición angular del péndulo (θ) .

Considerando l como la distancia desde el eje de rotación hasta el centro de masas del péndulo, sus coordenadas son:

$$\begin{cases} x_{cm} = x + l \sin(\theta) \\ y_{cm} = l \cos(\theta) \end{cases} \quad (2.3)$$

Esta elección de coordenadas establece de forma implícita el marco de trabajo del controlador. Se define la vertical superior como el origen angular $(\theta = 0 \text{ rad})$ y de acuerdo con la geometría de la Ecuación 2.3, se asume una convención de signos donde el sentido de giro horario se considera positivo para el incremento de θ .

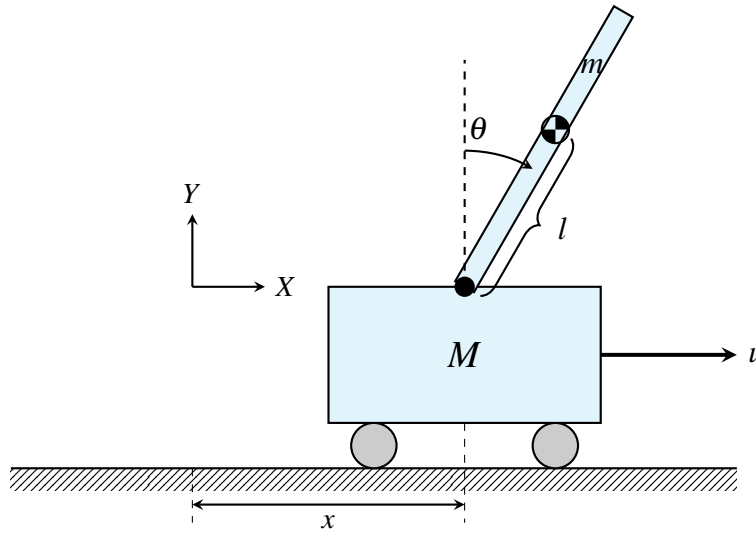


Figura 2.2: Esquema cinemático, definición del sistema de coordenadas

Una vez establecida la convención de signos del sistema y tener una visión esquemática ofrecida en la Figura 2.2, es momento de desarrollar la Ecuación 2.1 hasta obtener la forma matricial compacta de la Ecuación 2.2. La energía potencial y cinética del sistema en concreto se puede representar como [3]:

$$\begin{cases} E_c = \frac{1}{2}(M + m)\dot{x}^2 + m\dot{x}\dot{\theta}l \cos(\theta) + \frac{1}{2}ml^2\dot{\theta}^2 \\ E_p = mgl \cos(\theta) \end{cases} \quad (2.4)$$

A partir de formular la energía cinética y potencial, se desarrolla el Lagrangiano (Anexo A) y se calculan sus derivadas parciales hasta obtener las siguientes ecuaciones de movimiento [3]:

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + ml^2\ddot{\theta} - mgl\sin(\theta) = 0 \end{cases} \quad (2.5)$$

Antes de compactar las ecuaciones en la forma matricial de la Ecuación 2.2 es importante añadir los términos de rozamiento viscoso para el riel y el péndulo.

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) + c_x\dot{x} - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + ml^2\ddot{\theta} + c_\theta\dot{\theta} - mgl\sin(\theta) = 0 \end{cases} \quad (2.6)$$

Finalmente se compacta la expresión en forma matricial como indica la Ecuación 2.2. La caracterización del sistema que posteriormente se implementará en simulación es:

$$\begin{bmatrix} M+m & ml\cos\theta \\ ml\cos\theta & I+ml^2 \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} + \begin{bmatrix} c_x & -ml\dot{\theta}\sin\theta \\ 0 & c_\theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ -mgl\sin\theta \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} u \quad (2.7)$$

Si se analiza la Ecuación 2.7 se deducen tres características clave del sistema:

- **Sistema subactuado:** La acción de control u solo permite empujar el carro de forma horizontal, no hay control directo en el posicionado del péndulo.
- **Acoplamiento de términos:** El movimiento del carro afecta a la posición del péndulo y a su vez el movimiento del péndulo actúa como una perturbación en el posicionado del carro.
- **Sistema inestable:** Si se analiza el sistema en $\theta = 0 \text{ rad}$ presenta polos con parte real positiva debido al término $-mgl\sin\theta$. Por tanto, se requiere aplicar acciones de control en bucle cerrado para poder permanecer en esta posición.

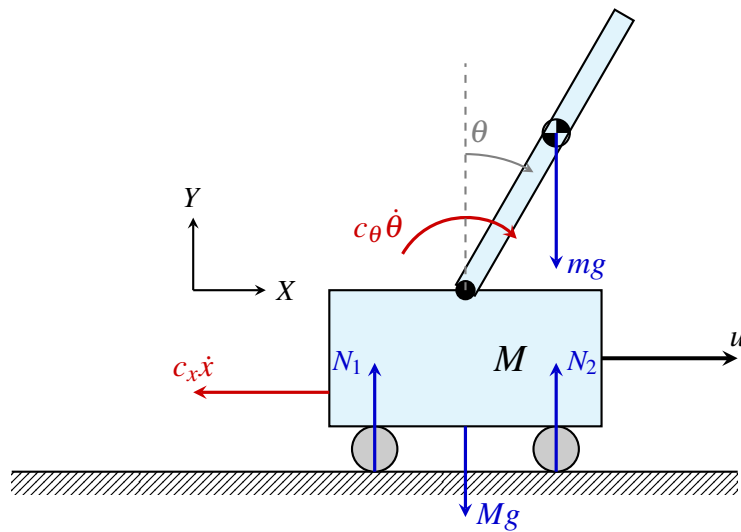


Figura 2.3: Descomposición de fuerzas

2.3.3 Estrategia de control Swing-up

La primera ley de control se centra en el levantamiento del péndulo desde su posición estable ($\theta = \pi \text{ rad}$) hasta su posición inestable y punto en el que se conmuta al control de posición ($\theta = 0 \text{ rad}$).

La idea intuitiva tras las matemáticas es simple, consiste en balancear el péndulo inyectando energía a favor del movimiento. El objetivo es llegar al punto de estabilización con pequeñas acciones de control que producen oscilaciones de amplitud creciente en el sistema.

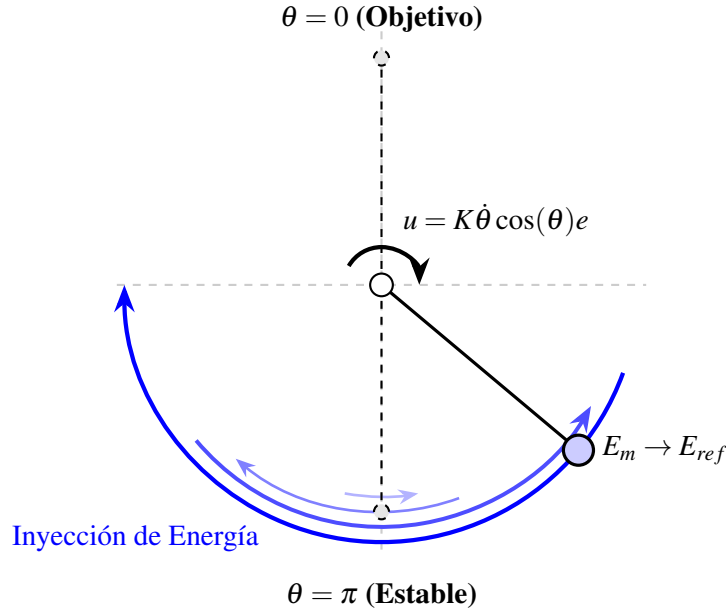


Figura 2.4: Esquema del control por balanceo (swing-up) basado en energía.

Para formalizar la ley de control esquematizada en la Figura 2.4, se define la energía del péndulo libre que viene dada por la suma de la energía potencial y la cinética [4]:

$$E_m = E_c + E_p = \frac{1}{2}J\dot{\theta}^2 + mgl \cos(\theta) \quad (2.8)$$

En el caso de la Ecuación 2.8 se define la máxima energía potencial cuando $\theta = 0 \text{ rad}$ de forma que el sistema de referencia del control coincide con el del modelado. Una vez se formaliza el modelo energético, se determina la referencia energética que el control por balanceo debe conseguir [4]:

$$E_{ref} = mgl \quad (2.9)$$

Por otro lado, el error de medida para calcular la acción de control u viene dado por la diferencia entre la energía actual del péndulo (E_m , Ecuación 2.8) y la energía de referencia (E_{ref} , Ecuación 2.9) [4]:

$$e = E_m - E_{ref} \quad (2.10)$$

Finalmente se define la ley de control que permite balancear el péndulo simple hasta su posición inestable, donde la acción de control u es el par aplicado en el eje de rotación [4]:

$$u = K\dot{\theta} \cos(\theta)e \quad (2.11)$$

En función del valor de la ganancia K se conseguirá el objetivo en diferente número de oscilaciones. Valores pequeños de la ganancia requieren de más iteraciones para llegar hasta el punto de estabilización.

Sin embargo, la ley de control definida en la Ecuación 2.11 no es directamente transferible al péndulo invertido sobre carro. Los motivos son los siguientes:

- **Acción de control:** En la ley de control propuesta en 2.11 el actuador se encuentra en el eje del péndulo. El sistema definido en 2.7 es subactuado y solo se tiene control sobre la fuerza de empuje del carro. Para aplicar la estrategia de energía, es necesario convertir la acción de control virtual (el par necesario en el eje) a una aceleración real del carro a través del acoplamiento dinámico del sistema.
- **Límites físicos del riel:** Se hace indispensable, por tanto, un control multiobjetivo que regule la energía del péndulo garantizando simultáneamente la estabilidad de la posición del carro dentro del espacio de trabajo del riel.

Para abordar las limitaciones mencionadas, se recurre a la técnica de **Linearización por Realimentación Parcial (PFL)**[5]. Permite transformar linealizar la dinámica de un subconjunto del sistema mediante un cambio de variables y una ley de control no lineal, siempre que exista un acoplamiento inercial suficiente entre las coordenadas actuadas y las pasivas [5].

En este trabajo se aplica la variante conocida como **linealización colocada** (*Collocated Linearization*), la cual consiste en linealizar el grado de libertad activo del sistema, es decir, aquel donde se ubica físicamente la acción de control [5]. El objetivo es diseñar una ley de control $u(N)$ que cancele las no linealidades de la dinámica del carro para que este se comporte, desde el punto de vista del controlador de alto nivel, como un doble integrador respecto a una aceleración deseada $\ddot{x}_{des}(\frac{m}{s^2})$.

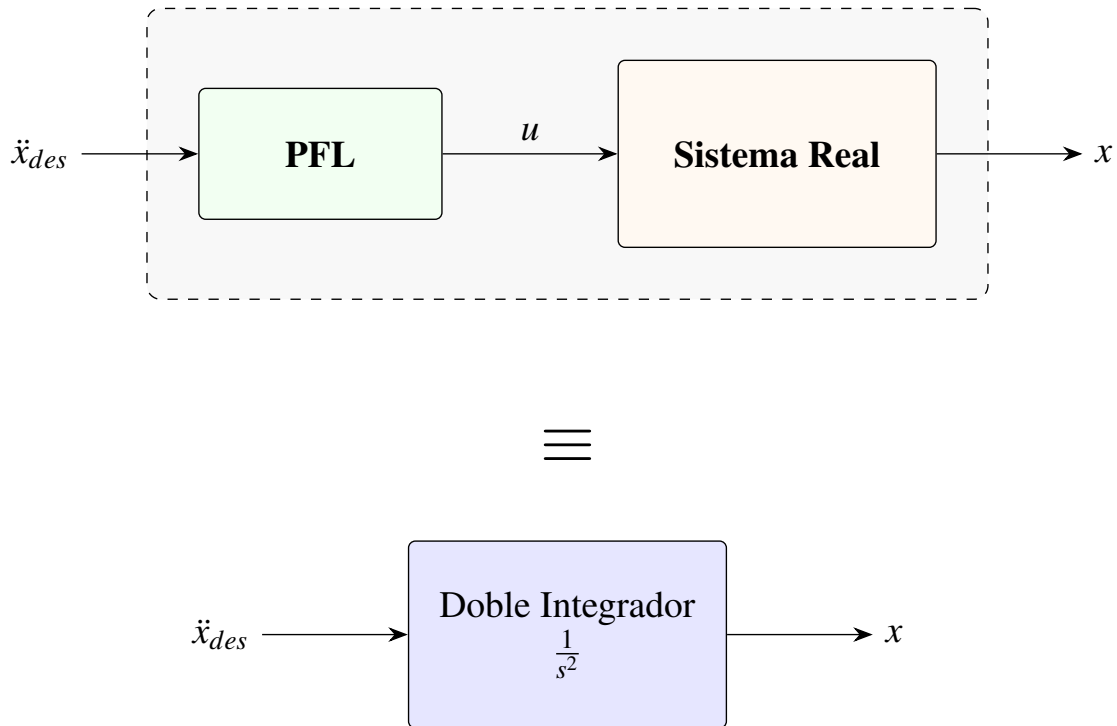


Figura 2.5: Concepto de equivalencia de la Linealización Colocalizada.

Partimos de las ecuaciones de movimiento del sistema que incluyen los términos de fricción del carro (c_x) y del péndulo (c_θ):

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) + c_x\dot{x} - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + J\ddot{\theta} + c_\theta\dot{\theta} - mgl\sin(\theta) = 0 \end{cases} \quad (2.12)$$

Donde $J = I + ml^2$ es el momento de inercia del péndulo respecto al eje de rotación. Para eliminar $\ddot{\theta}$ de la ecuación de control, se despeja dicha variable de la dinámica del péndulo [3]:

$$\ddot{\theta} = \frac{1}{J} (mgl\sin\theta - ml\cos\theta\ddot{x} - c_\theta\dot{\theta}) \quad (2.13)$$

Al imponer la condición de linealización $\ddot{x} = \ddot{x}_{des}$ y sustituir la expresión 2.13 en la ecuación del carro, se obtiene la fuerza u necesaria para anular la dinámica no lineal del subsistema actuado [3]:

$$u = (M+m)\ddot{x}_{des} + ml\cos\theta \left[\frac{1}{J} (mgl\sin\theta - ml\cos\theta\ddot{x}_{des} - c_\theta\dot{\theta}) \right] - ml\dot{\theta}^2\sin\theta + c_x\dot{x} \quad (2.14)$$

Como se observa, la fuerza física aplicada (u) compensa dinámicamente la masa total y los efectos inerciales del péndulo. Esta arquitectura permite definir la aceleración deseada $\ddot{x}_{des}$ como el punto de encuentro donde convergen los dos objetivos de control del sistema [3]:

$$\ddot{x}_{des} = \underbrace{K_e\dot{\theta}\cos\theta(E - E_{ref})}_{\text{Control de Energía (Swing-up)}} - \underbrace{(K_px + K_d\dot{x})}_{\text{Control de Posición (PD)}} \quad (2.15)$$

La ley de control resultante en la ecuación 2.15 simboliza un compromiso entre la inyección de energía necesaria para el balanceo y la regulación de la posición del carro sobre el riel [3]. Se debe ajustar el peso de cada ganancia para obtener un equilibrio óptimo entre ambos controles.

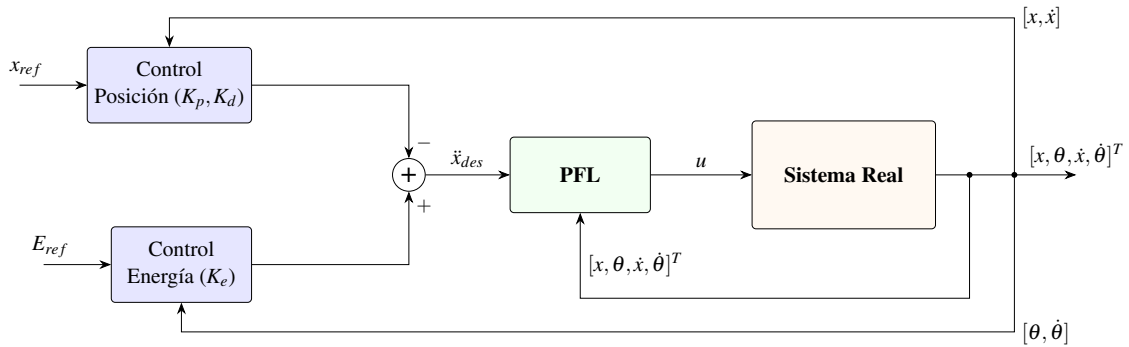


Figura 2.6: Diagrama de bloques de la estrategia de Swing-up con linealización parcial (PFL)

2.3.4 Modelado en el espacio de estados

Una vez que la estrategia de *swing-up* ha llevado al péndulo a su posición vertical inestable ($\theta \approx 0 \text{ rad}$), se requiere mantener el péndulo equilibrado en su punto superior mientras se regula la posición del carro en un punto de referencia (típicamente el centro del riel, $x = 0$). Para lograr el cometido se opta por aplicar un control por realimentación del estado. Un control de este tipo, requiere una representación en el espacio de estados del modelo dinámico linealizado en el punto de equilibrio. Todos los sistemas bajo este marco se definen como [6]:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (2.16)$$

Donde el vector de estados $\mathbf{x}(t) = [x, \theta, \dot{x}, \dot{\theta}]^T \equiv [q, \dot{q}]^T$ define completamente la condición del sistema en cada instante. Los componentes de la representación se describen a continuación [6]:

- **Matriz de estado** ($A \in \mathbb{R}^{4 \times 4}$): Caracteriza la dinámica interna del sistema y cómo evolucionan los estados (posiciones y velocidades) entre sí cuando no hay acción de control. Sus autovalores son los polos del sistema e indican si es estable.
- **Matriz de entrada** ($B \in \mathbb{R}^{4 \times 1}$): Define cómo la fuerza aplicada al carro (u) afecta a la aceleración de cada uno de los estados.
- **Matriz de salida** ($C \in \mathbb{R}^{4 \times 4}$): Relaciona los estados internos con las variables medidas. Dado que en este proyecto se dispone de sensores para todas las variables, C se simplifica a la matriz identidad I .
- **Matriz de transmisión directa** (D): Representa el acoplamiento directo entre la entrada y la salida. Para este caso D es cero, ya que la acción de control actúa sobre la dinámica del sistema primero.

Para sistematizar la transición entre el modelo físico no lineal y el diseño del controlador de estabilidad, se parte de la ecuación 2.2. Al aplicar una linealización por Jacobiano alrededor del punto de equilibrio inestable ($\bar{q}$ con $\dot{\bar{q}} = 0$), las matrices de la dinámica se transforman en matrices de coeficientes constantes evaluadas en dicho punto [6]:

- **Inercia:** $\bar{M} = M(\bar{q})$
- **Amortiguamiento:** $\bar{C} = C(\bar{q}, 0)$
- **Rigidez (Gravedad):** $\bar{G} = \left. \frac{\partial G(q)}{\partial q} \right|_{q=\bar{q}}$
- **Entrada:** $\bar{B} = B(\bar{q})$

Para el caso en particular del péndulo invertido sobre carro, el sistema linealizado en dicho punto es el siguiente:

$$\underbrace{\begin{bmatrix} M+m & ml \\ ml & I+ml^2 \end{bmatrix}}_{\bar{M}} \underbrace{\begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix}}_{\ddot{\mathbf{q}}} + \underbrace{\begin{bmatrix} c_x & 0 \\ 0 & c_\theta \end{bmatrix}}_{\bar{C}} \underbrace{\begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix}}_{\dot{\mathbf{q}}} + \underbrace{\begin{bmatrix} 0 \\ -mgl \end{bmatrix}}_{\bar{G}} = \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\bar{B}} u \quad (2.17)$$

El sistema descrito en la ecuación 2.17 es bajo el que se diseñará el control de posición. Contra más lejos esté el péndulo de su vertical menos efectivo será el control, ya que las diferencias del

sistema real respecto del lineal empiezan a acentuarse, este es uno de los motivos por el que se hace necesaria la conmutación entre control de posición y swing-up.

Definiendo el vector de estados como $\mathbf{x} = [q, \dot{q}]^T$, el sistema puede reescribirse en la representación compacta de espacio de estados mediante la siguiente expresión [6]:

$$\begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{0} & I \\ -\bar{M}^{-1}\bar{G} & -\bar{M}^{-1}\bar{C} \end{bmatrix}}_A \begin{bmatrix} q \\ \dot{q} \end{bmatrix} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \bar{M}^{-1}\bar{B} \end{bmatrix}}_B u \quad (2.18)$$

1. Planta Real

2. Linealización

3. Abstracción

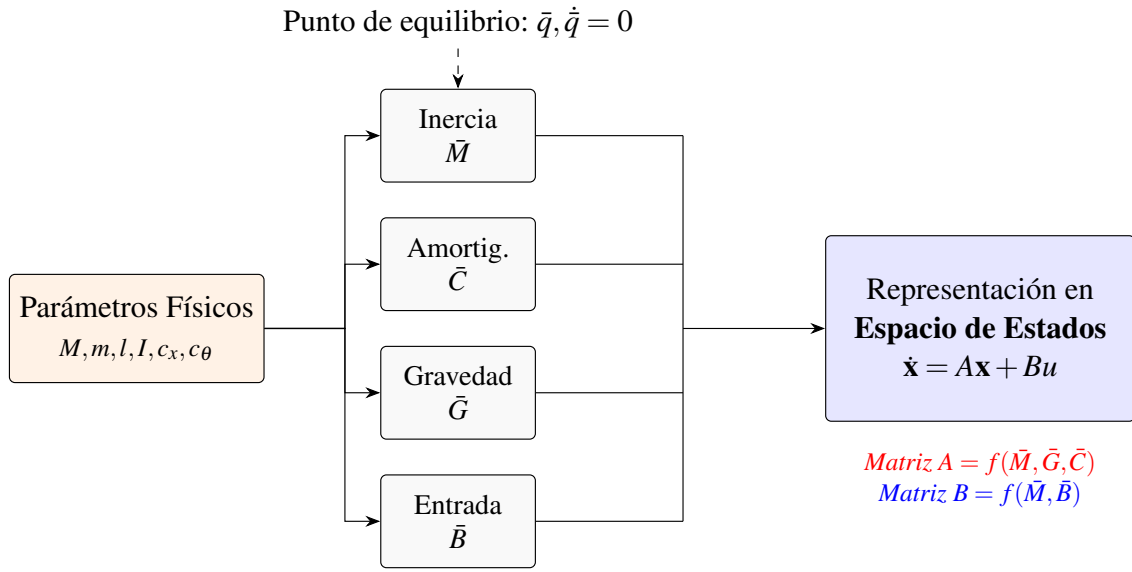


Figura 2.7: Flujo de trabajo para el modelado

Las principales ventajas de aplicar esta técnica para la obtención de la representación del sistema en el espacio de estados son:

- **Abstracción y generalización del modelado:** La técnica es escalable a cualquier sistema que cumpla la dinámica definida en la ecuación 2.2. Permite centrarse en el diseño del controlador en lugar de obtener de forma manual las matrices A y B para cada sistema concreto.
- **Eficiencia en la implementación:** La transición a un marco matricial facilita enormemente la implementación en lenguajes de programación de alto nivel, como *MATLAB* o *Python* (mediante la librería *NumPy*). El programador simplemente define las matrices ($\bar{M}$, $\bar{C}$, $\bar{G}$ y $\bar{B}$) luego el propio lenguaje obtiene en la salida la representación en espacio de estados (A y B).

2.3.5 Control por realimentación del estado

Tras definir en la anterior sección el *sistema lineal invariante (LTI)* que define la dinámica del sistema en el punto de equilibrio ($q = 0, \dot{q} = 0$), se requiere diseñar una ley de control que permita estabilizar el sistema en esta región.

El **control por realimentación del estado** para este caso asume que todos los estados del sistema ($\mathbf{x}$) están disponibles para su medición. Bajo esta premisa, la acción de control $u(t)$ se calcula como una combinación lineal de dichos estados multiplicados por un vector de ganancias K , sumado opcionalmente a un término de referencia $r(t)$ escalado por una ganancia K_r [7]:

$$u(t) = K_r r(t) - K \mathbf{x}(t) \quad (2.19)$$

Desglosando el vector de ganancias $K = [K_1, K_2, K_3, K_4]$, la fuerza aplicada al carro se define de forma extendida como:

$$u = K_r r - (K_1 x + K_2 \theta + K_3 \dot{x} + K_4 \dot{\theta}) \quad (2.20)$$

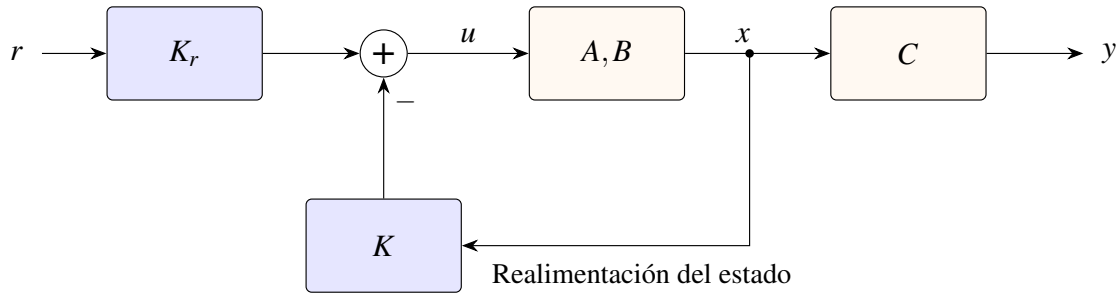


Figura 2.8: Diagrama de bloques de la ley de control por realimentación del estado

La figura 2.8 muestra como el controlador obtiene la acción de control a aplicar en cada instante de tiempo en función de los estados y la referencia. Por otro lado, en comparación con la ecuación 2.16, el sistema en bucle cerrado representado en espacio de estados es [7]:

$$\begin{cases} \dot{x}(t) = Ax(t) + B(K_r \cdot r(t) - K \cdot x(t)) = (A - BK)x(t) + BK_r r(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (2.21)$$

Bajo esta configuración, la dinámica del sistema en bucle cerrado queda gobernada por la expresión matricial $(A - BK)$. Por tanto, el diseño del controlador se reduce a determinar los valores del vector de ganancias $K = [k_1, k_2, k_3, k_4]$ que garanticen que todos los autovalores resultantes posean parte real negativa, dotando así al sistema de una respuesta temporal específica. Para abordar este problema, se consideran habitualmente dos metodologías:

- **Asignación de polos:** Consiste en definir arbitrariamente la ubicación de los autovalores de la matriz $(A - BK)$ en el plano complejo. Se calculan las ganancias de K para que el sistema responda con una dinámica concreta (amortiguamiento y tiempo de establecimiento) [7].

- **Regulador Cuadrático Lineal (LQR):** Esta metodología aborda el diseño como un problema de optimización. En lugar de elegir la ubicación de los polos, el diseñador define qué es más costoso: que el sistema se aleje del equilibrio o que el actuador trabaje en exceso [7]. El algoritmo calcula de forma automática la matriz de ganancias K que minimiza el siguiente índice de coste cuadrático:

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt \quad (2.22)$$

El comportamiento del controlador se sintoniza a través de dos matrices de ponderación:

- **Matriz Q (4×4):** Penaliza la desviación de los estados. Valores elevados en la diagonal de Q indican que se prioriza la precisión y la rapidez en la estabilización de los estados $(x, \theta, \dot{x}, \dot{\theta})$.
- **Matriz R (1×1):** Penaliza el esfuerzo de control $u(t)$. Un valor de R elevado suaviza la respuesta del actuador.

Es importante notar que la respuesta del sistema no depende de los valores absolutos de las matrices, sino del peso relativo entre ellas (Q frente a R).

2.3.6 Lógica de conmutación

Finalmente, la lógica de conmutación atiende a un umbral angular. Este bloque de decisión monitoriza de forma continua la posición angular estimada del péndulo ($\hat{\theta}$). Si el péndulo ingresa dentro de una vecindad acotada en torno a la vertical absoluta (*el punto de linealización del sistema*), la lógica conmuta al control por realimentación del estado. En caso contrario, el sistema mantiene activo el controlador de swing-up para elevar el mecanismo. Este comportamiento híbrido se formaliza en el siguiente pseudocódigo:

Algoritmo 1 Lógica de Conmutación

Require: Vector de estados estimado $\hat{x} = [\hat{x}_c, \dot{\hat{x}}_c, \hat{\theta}, \dot{\hat{\theta}}]^T$, umbral angular θ_{th}

Ensure: Señal de control u a aplicar a la planta

```

1: loop
2:    $\hat{\theta} \leftarrow \hat{x}(3)$                                 ▷ Obtener posición angular actual
3:   if  $|\hat{\theta}| \leq \theta_{th}$  then
4:      $u \leftarrow -K\hat{x}$                                 ▷ Control de posición
5:   else
6:      $u \leftarrow f_{swing}(\hat{x})$                         ▷ Control de energía
7:   end if
8:
9: end loop

```

2.4 Arquitectura del sistema

Para definir la arquitectura del sistema se ha hecho una división entre la parte de simulación y la implementación en un sistema real.

La arquitectura del sistema de simulación permite obtener una vista general del flujo de trabajo para el modelado y la validación antes de la implementación real. Por otro lado, la arquitectura del sistema real se centra en los periféricos involucrados en el control así como el proceso de discretización que se debe aplicar para controlar un sistema real utilizando un microcontrolador.

2.4.1 Arquitectura de la simulación

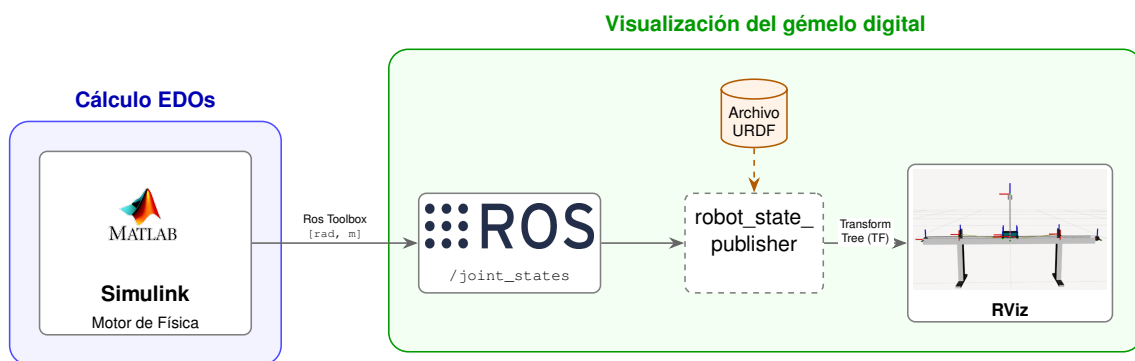


Figura 2.9: Arquitectura de la simulación

La Figura 2.9 detalla la arquitectura de software diseñada para la simulación del sistema. A continuación se desglosa cada elemento del esquema para conocer su función y como se conecta con el resto de la arquitectura:

- **Cálculo de EDOs (Motor de Física):** Implementado en el entorno MATLAB/Simulink, este bloque se encarga de resolver las Ecuaciones Diferenciales Ordinarias (EDO) que describen la dinámica del sistema. Simulink actúa como el motor de física, calculando en cada paso de tiempo los estados del robot (posiciones en radianes y metros). Estos datos se transmiten al ecosistema ROS mediante ROS Toolbox, publicando la información en el tópico `/joint_states`.
- **Visualización, gemelo digital:** Este bloque recibe la información cinemática para representar el comportamiento del sistema en tiempo real. Está totalmente integrado en el entorno de ROS y se compone de varias piezas:
 - **Archivo URDF (Unified Robot Description Format):** Es un archivo XML que describe como es el robot o mecanismo que se quiere simular. Se especifican las visuales, las restricciones cinemáticas y los marcos de referencia de las articulaciones.
 - **Robot State Publisher:** Es el nodo de ROS que permite representar el movimiento del robot. Se suscribe al tópico `/joint_states` y actualiza el árbol de transformadas de nuestro archivo URDF.
 - **RViz:** La última pieza de la arquitectura software, es la interfaz que permite visualizar de forma gráfica e intuitiva todo el flujo de datos como un modelo 3D en movimiento.

2.4.2 Arquitectura del sistema real

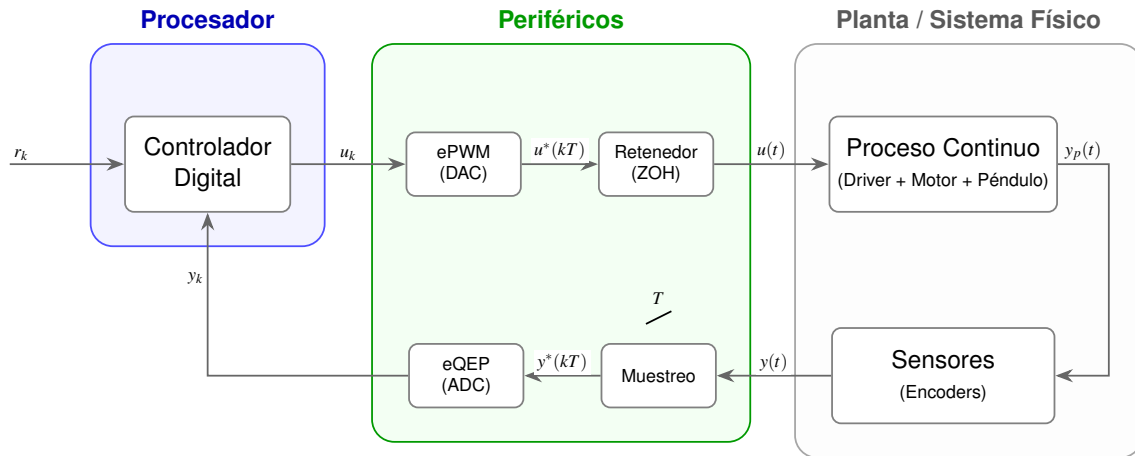


Figura 2.10: Arquitectura del sistema real

La Figura 2.10 presenta de forma esquemática la implementación del bucle de control en un microcontrolador.

Esta estructura permite observar la interacción entre el entorno digital (discreto) y el proceso físico (continuo). El núcleo del sistema es el **Controlador Digital**. Este procesador se encarga de ejecutar el algoritmo de control que procesa la señal de error entre la referencia (r_k) y la realimentación (y_k).

El concepto fundamental es el **periodo de muestreo** (T). Este parámetro determina la periodicidad de la rutina de control, permitiendo sincronizar las etapas de medida, cálculo y actuación sobre la planta de manera precisa. Durante el desarrollo del proyecto se determinará el periodo de muestreo atendiendo a factores como la dinámica del sistema en bucle cerrado o la frecuencia de la señal PWM.

Finalmente la interacción entre el mundo digital y el mundo físico es posible mediante el uso de **periféricos** que permiten medir el estado del sistema o modificarlo. Diferenciamos en dos clases de periféricos:

- **Actuadores:** La señal de control digital (u_k) se convierte en una señal analógica o cuasi-analógica mediante el módulo *ePWM* (DAC, convertidor digital-analógico). Posteriormente, un bloque *Retenedor de orden cero* (ZOH) mantiene la señal constante durante el intervalo de muestreo, transformándola en la señal continua $u(t)$ que excita al Proceso Continuo (compuesto por el driver, el motor y el péndulo).
- **Sensores:** La respuesta del sistema ($y_p(t)$) es captada por los sensores (en este caso, encoders). La señal $y(t)$ es procesada por un bloque de muestreo, gobernado por el periodo T , y enviada al periférico *eQEP*. Este último actúa como un equivalente a un ADC (convertidor analógico-digital), transformando los pulsos del encoder en una señal digital y_k interpretable por el controlador.

2.4.3 Especificación de componentes de hardware

Para la implementación física del sistema de control diseñado, se han seleccionado componentes que garantizan un compromiso óptimo entre precisión de lectura, capacidad de procesamiento y tiempo de respuesta. En la Tabla 2.1 se resumen los elementos principales del prototipo.

Cuadro 2.1: Resumen de componentes del sistema de hardware.

Componente	Modelo	Función Principal
Microcontrolador	TMS320F280049C	Procesamiento de señales y ejecución del control
Driver de motor	[Modelo, ej. L298N]	Interfaz de potencia (Puente H)
Encoder Motor	[Modelo/PPR]	Medición de posición lineal (x)
Encoder Péndulo	[Modelo/PPR]	Medición de ángulo vertical (θ)
Actuador	Motor DC [V]	Generación de fuerza sobre el carro (u)

Unidad de Procesamiento (Microcontrolador)

Para la implementación del algoritmo de control se ha seleccionado el **TMS320F280049C**, un microcontrolador de tiempo real de la familia **C2000** de **Texas Instruments**. Este dispositivo está específicamente diseñado para aplicaciones de control, destacando su núcleo **C28x** a **100 MHz** que integra una unidad de punto flotante (**FPU**) y una unidad de aceleración trigonométrica (**TMU**) [8]. Ambos aceleradores son fundamentales para procesar la aritmética del controlador y las funciones trigonométricas del modelo.

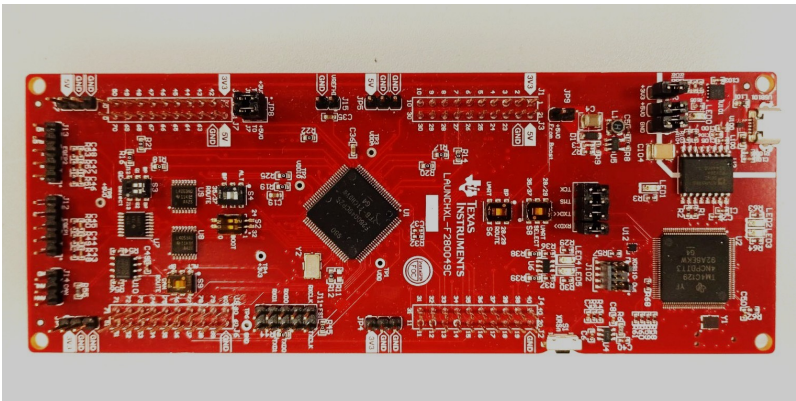


Figura 2.11: TMS320F280049C en formato *Launchpad*

Asimismo, el chip dispone de periféricos de control avanzado que descargan de tareas críticas a la CPU, los más destacados en el proyecto son:

- **eQEP (Enhanced Quadrature Encoder Pulse):** Dos módulos dedicados para la decodificación de señales en cuadratura de los encoders, permitiendo una lectura de posición y ángulo sin consumo de ciclos de reloj [8].
- **ePWM (Enhanced Pulse Width Modulator):** Capaz de generar señales con una resolución de hasta **150 ps**, garantizando un control fino sobre el motor DC [8].

El desarrollo software se apoya en el ecosistema oficial de Texas Instruments. Como entorno de desarrollo integrado (IDE) se emplea **Code Composer Studio (CCS)** [9], permite una depuración avanzada y optimización específica para la arquitectura C28x. Además, se utiliza la **SDK C2000Ware**, que proporciona una capa de abstracción de hardware (HAL) facilitando la programación y la interpretabilidad del código [10].

Etapas de Potencia y Actuación

La acción de control $u(t)$ se traduce en una tensión aplicada al motor mediante el driver de motores de corriente continua **AQMH2407ND**. Implementa un **Puente H** de doble canal basado en tecnología **MOSFET** [11], lo que le otorga una eficiencia energética superior y una disipación térmica significativamente menor en comparación con drivers basados en transistores **BJT**, como el **L298N** [12].

El dispositivo es capaz de suministrar una corriente continua de hasta **7 A** por canal (con picos de hasta 50 A) [11]. El control se gestiona mediante señales PWM provenientes del microcontrolador, permitiendo la regulación de la tensión media aplicada al motor.

Un aspecto fundamental en el diseño de este driver es la integración de **optoacopladores EL357N-G** [13], los cuales proporcionan un **aislamiento galvánico** entre la etapa de control y la de potencia. Esta característica es esencial en la implementación de sistemas de control, ya que ofrece:

- **Mitigación de ruido:** Evita que el ruido electromagnético inducido por la conmutación de las bobinas del motor afecte a las señales de medición.
- **Protección del hardware:** Asegura la integridad del microcontrolador frente a posibles picos de tensión o corrientes de retorno (*back-EMF*) provenientes de la etapa de potencia.

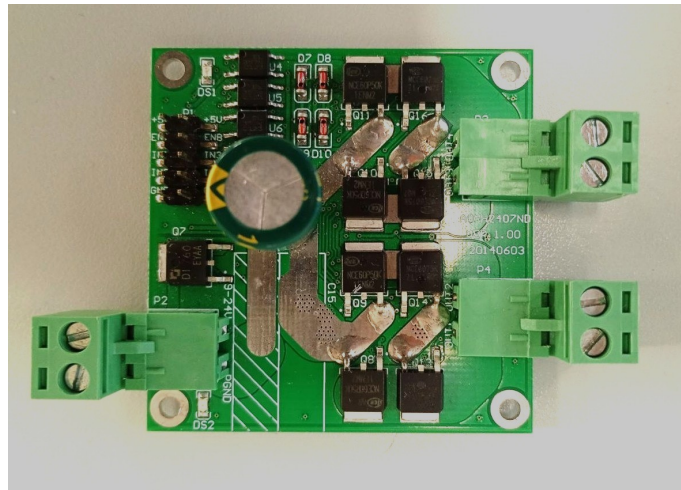


Figura 2.12: Driver AQMH2407ND

Sensores de Estado (Encoders)

La precisión en la estimación de la velocidad ($\dot{x}$, $\dot{\theta}$) depende directamente de la resolución de los sensores:

- **Posición del carro:** Se utiliza un encoder acoplado a [eje motor/correa] que proporciona [X] pulsos por revolución, permitiendo una resolución milimétrica.
- **Ángulo del péndulo:** Se emplea un encoder incremental de alta resolución ([X] PPR) situado en el eje de rotación. Se ha prestado especial atención a que este sensor presente una fricción mínima, para no invalidar las asunciones del modelo dinámico ideal sin rozamiento.

3. Desarrollo

Explicación de las actividades realizadas durante el desarrollo del proyecto. Se puede exponer en orden cronológico o por grupos de tareas. Hay que indicar los hitos alcanzados.

En esta sección solamente hay que aportar la información técnica estrictamente necesaria para comprender el trabajo realizado. Esta información debe, además, estar presentada de forma que sea inteligible por cualquiera no familiarizado con los aspectos técnicos del trabajo realizado. La información técnica puede ubicarse en los apéndices para ser consultada si se desea.

3.1 Simulación

1) Uso de parámetros proporcionados por el fabricante (rozamiento, inercias, etc.) para modelado del sistema en Matlab/Simulink.

2) Conexión exitosa entre ROS y Matlab mediante la herramienta ROS Toolbox, hay que comentar como se tubo que cambiar el DDS al cyclone para funcionar, la lógica de publicador subscriber, el diseño de un archivo URDF para la visualización de la unidad mecánica desde RViz. (Posibles vías de estudio como la migración de la simulación completamente a ROS)

3) Explicar también como mediante simulación se pudo obtener las ganancias necesarias para poder hacer control swin-up con PFL. Muy importante poner los diagramas de bloques de nuestro simulink (El de publicación de tópicos, el de dinámica del sistema y el de conmutación de control (Swing-Up to LQR))

4) Finalmente explicar como se acabo implementando una simulación discreta del sistema (uso de bucle for y mida de paso con el método de Euler o1). Sirve como puente a la implementación de algoritmos de control en microcontroladores.

3.2 Sistema físico

1) Selección de microcontrolador, Pte H y encoders. Explicar un poco la tecnología detrás de cada componente (Ejemplo: Sabemos que los encoders son HEDS-9140-A00, también sabemos que el PTE H tiene aislamiento galvánico mediante optoacopladores).

2)Firmware, comentar como se programa una DSP de texas instruments mediante el uso del IDE CCS y la capa de HAL C2000-ware. Utilizar el datasheet del fabricante para justificar la configuración de ciertos registros. Módulos Epwm y Eqp muy interesante ver como liberan a la CPU de estas tareas.

3)Explicar como se programo la DSP con timers para ejecutar el bucel de control en una rutina de interrupción periódica (implica determinismo, nada del loop de Arduino)

4)(Lo que hay que acabar de hacer este més) Razonar experimentos de identificación de parámetros del sistema. La justificación es que los parámetros del fabricante no sabemos si son del todo ciertos al ser una máquina de hace 25 años. Algunos experimentos: Conversión de voltios a fuerza horizontal (encontrar la constante que relaciona ambas magnitudes), Identificación del rozamiento horizontal del carro, identificación del rozamiento y la inercia del péndulo.

5)Simular de nuevo con los parámetros obtenidos mediante identificación.

Para la configuración de los periféricos se ha optado por el uso de estructuras de campos de bits (Bit-fields) proporcionadas por la SDK *C2000Ware*. A diferencia de la librería de abstracción *DriverLib*, este enfoque permite un acceso directo y optimizado a los registros del procesador mediante el uso de punteros a estructuras mapeadas en memoria. Esta elección garantiza la máxima eficiencia en la ejecución, reduciendo la sobrecarga de funciones (*overhead*) y facilitando el control preciso de registros críticos como los de los módulos *eQEP* y *ePWM*.

4. Resultados

- 4.1 Resultados en simulación
- 4.2 Resultados en sistema físico

5. Conclusiones y posible trabajo futuro

Trabajo a futuro → Implementación en HAL, uso de algoritmos no basados en modelo (Reinforcement Learning)

Bibliografía

- [1] M. W. Spong, S. Hutchinson y M. Vidyasagar, *Robot modeling and control*. John Wiley & Sons, 2006 (véase página 10).
- [2] R. M. Murray, Z. Li y S. S. Sastry, *A mathematical introduction to robotic manipulation*. CRC press, 1994 (véase página 10).
- [3] R. Tedrake, *Underactuated Robotics: Algorithms for Planning and Control*. MIT Press, 2023, Disponible online en <http://underactuated.mit.edu/> (véanse páginas 11, 15).
- [4] K. J. Åström y K. Furuta, “Swinging up a pendulum by energy control,” *Automatica*, volumen 36, páginas 287-295, 2000, Disponible online en <https://www.elsevier.com/locate/automatica> (véase página 13).
- [5] M. W. Spong, “Partial Feedback Linearization of Underactuated Mechanical Systems,” en *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, Coordinated Science Laboratory, University of Illinois at Urbana-Champaign, IEEE, San Diego, CA, mayo de 1994, páginas 2356-2361 (véase página 14).
- [6] I. P. Alós, *Tema 7 - Análisis en Espacio de Estados del comportamiento dinámico de sistemas*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véanse páginas 16, 17).
- [7] I. P. Alós, *Tema 8 - Control por realimentación del estado*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véanse páginas 18, 19).
- [8] *TMS320F28004x Real-Time Microcontrollers Data Sheet*, manual, Literature Number: SPRSAD2J. Disponible en <https://www.ti.com/lit/ds/symlink/tms320f280049c.pdf>, Texas Instruments, 2024 (véase página 22).
- [9] T. Instruments, *Code Composer Studio (CCS) Integrated Development Environment*, misc, Disponible en <https://www.ti.com/tool/CCSTUDIO>, 2025 (véase página 22).
- [10] T. Instruments, *C2000Ware Software Development Kit*, misc, Disponible en <https://www.ti.com/tool/C2000WARE>, 2025 (véase página 22).

- [11] *7A-160W Dual Channel DC Motor Drive Module User Manual*, manual, Modelo: AQMH2407ND. Disponible en <https://www.handsontec.com/dataspecs/module/7A-160W%20motor%20control.pdf>, HandsOn Technology, 2024 (véase página 23).
- [12] *L298 Dual Full-Bridge Driver Data Sheet*, manual, Disponible en <https://www.st.com/resource/en/datasheet/l298.pdf>, STMicroelectronics, 2000 (véase página 23).
- [13] *EL357N-G Series: 4 PIN SOP Phototransistor Photocoupler*, manual, Literature Number: DPC-0000014 Rev. 6, Everlight Electronics Co., Ltd., 2014 (véase página 23).

6. Apéndices

Colocar aquí cualquier tipo de información susceptible de ser consultada para la correcta comprensión del documento y el proyecto que describe.

A. Desarrollo del Lagrangiano

En este anexo se detalla el desarrollo matemático paso a paso del formalismo de Euler-Lagrange, partiendo de las energías del sistema hasta obtener la formulación matricial.